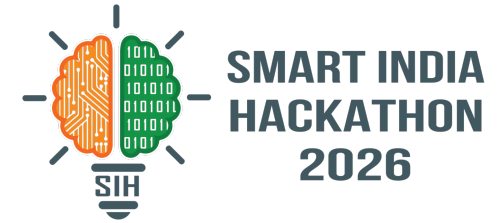


SMART INDIA HACKATHON 2026



JOCKY — a forensic scripting language for computer & network analysis

Problem Statement ID: [〈 fill from portal 〉](#)

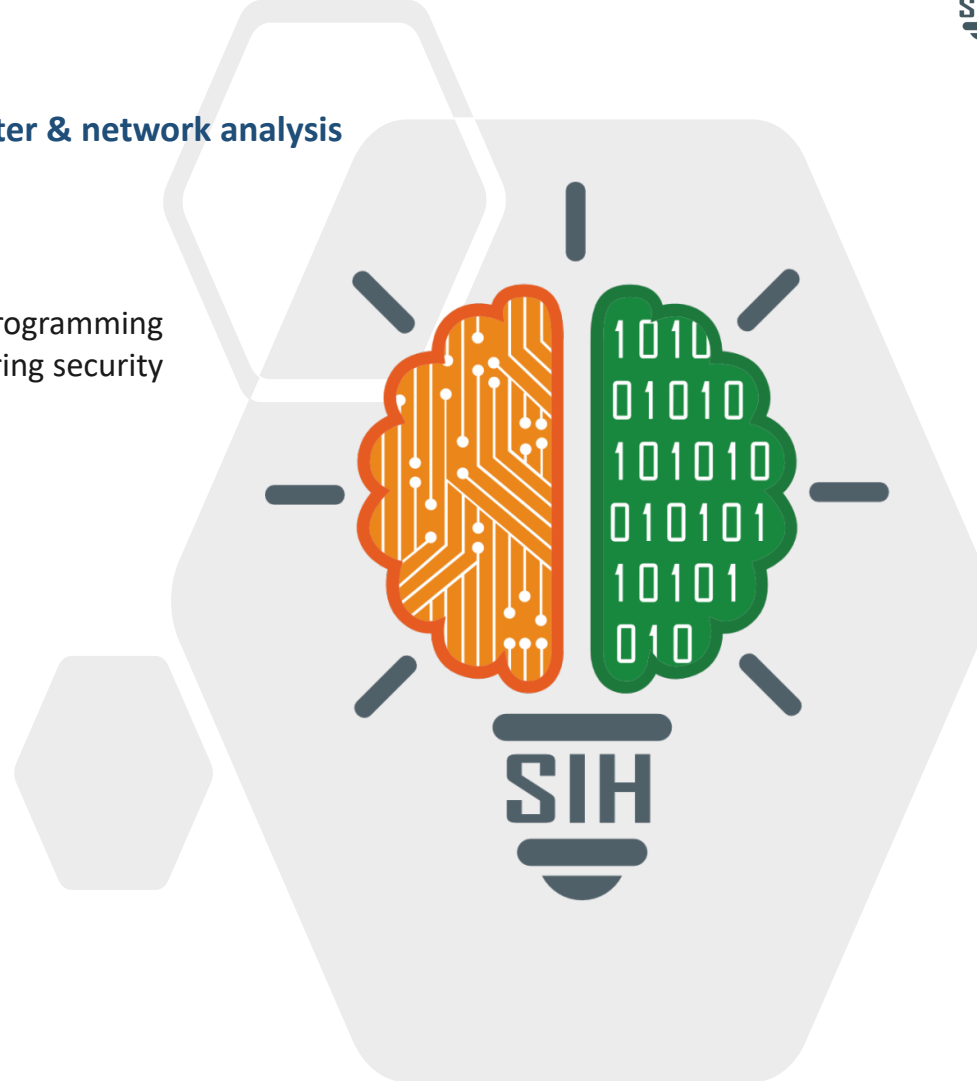
Problem Statement Title: Creation of scripts/functions with a new programming language to commence computer & network forensic analysis without triggering security solutions

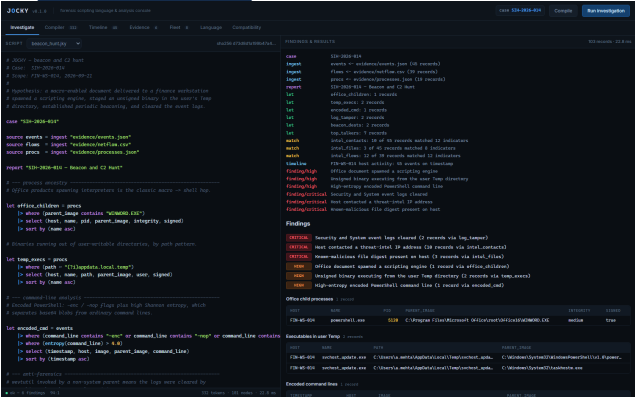
Theme: Blockchain & Cybersecurity

PS Category: Software

Team ID: [148458 〈 verify 〉](#)

Team Name (Registered on portal): Data_Drishti





Operator console — findings cite the query that produced them

INNOVATION & UNIQUENESS (IUP)

- Findings cannot be uncited. A finding must name the result set that raised it — provenance is enforced by the grammar, not by examiner discipline.
- Offline and dependency-free. Pure Python standard library. Runs on an air-gapped examiner workstation with nothing installed.
- Compatible by design, not by stealth. Signed reproducible builds with published hashes for vendor allowlisting — no kernel drivers, no injection, no log tampering.
- Judge-independent. Deterministic execution: the same script over the same digests yields the same findings, byte for byte.

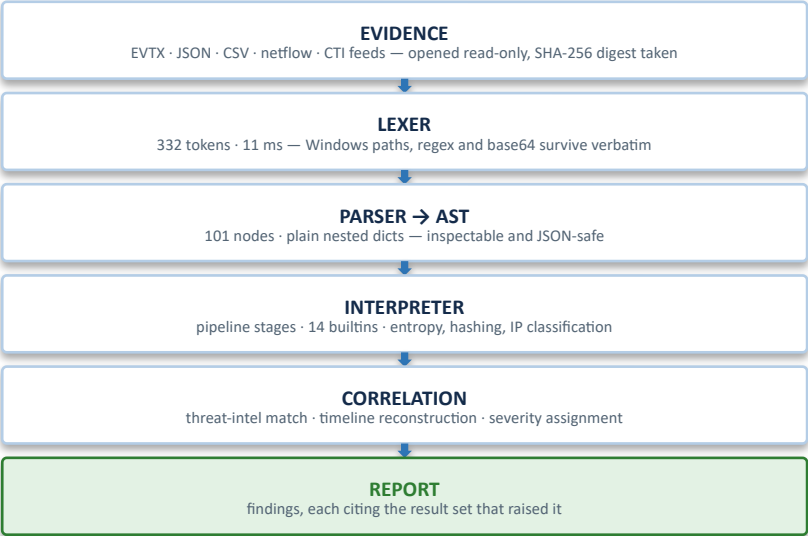
THE PROBLEM IT TACKLES

- Examiners stitch tools together. Volatility, plaso, KAPE, Chainsaw and throwaway scripts — every hop loses provenance.
- Findings arrive without a citable query. Ad-hoc pipelines produce results no one can reproduce or defend on the stand.
- Security tooling fights the endpoint. Collection and analysis trigger AV/EDR, and the industry habit is to weaken the defence instead of documenting the telemetry.
- The tooling becomes the risk. Anything that hides from endpoint security is indistinguishable from the intrusion it is meant to investigate — examiners are asked to trade visibility for capability.

PROPOSED SOLUTION

- One language, one evidence model. A pipeline DSL — source → |> where |> group by |> sort by — over host events, processes, netflow and CTI feeds alike.
- Every finding is a citation. A finding names the result set and record count behind it; the run logs source and script digests, so the report is reproducible.
- Runs with the endpoint, not against it. Read-only evidence access, loopback-only console, documented telemetry sources, signed builds — no evasion of any kind.
- Small enough to audit. Roughly 2,000 lines of Python across six modules with a 50-test suite — an examiner can read the whole interpreter before deciding to trust it.

GRAPHICAL ABSTRACT



no kernel drivers · no injection · no obfuscation · no log tampering

11 ms first finding on 45 records	0 third-party dependencies	6 / 6 findings cite their query	127.0.0.1 console binds loopback only
---	----------------------------------	---------------------------------------	---

WHY IT IS DIFFERENT

- Sigma detects, VQL queries — neither makes a finding citable. JOCKY's grammar refuses a finding that does not name its source.
- A DSL is a contract. Scripts become reviewable, versionable artefacts that transfer between examiners and agencies.

Technologies to be used (languages, frameworks, hardware) · Methodology and process for implementation

LAYERED ARCHITECTURE

Presentation	Operator console — one HTML file, 7 tabs, deep-linkable, Ctrl+Enter to run
Service	serve.py — loopback API (/api/state, /run, /compile), client-address check, path-traversal guard
Language core	lexer → parser → interpreter · AST as plain nested dicts · 11 ms for a 332-token script
Primitives	14 builtins — entropy, sha256, md5, is_public_ip, age_minutes, regex
Evidence adapters	JSON · CSV · line-based CTI — read-only, digested at open

STATE MACHINE — ONE SCRIPT RUN



REJECTED STATES — a run fails loudly, never silently

SYNTAX ERROR line:col with a caret under the offending token	UNKNOWN BINDING caught by 'lint' before evidence is read	RUNTIME ERROR missing evidence names the path it looked for
--	--	---

METHODOLOGY — DATA FLOW

*Evidence is never written to; every artefact is digested before it is parsed. A script that cites an undeclared binding fails 'lint' before it ever touches evidence.*

TECHNOLOGY STACK & LANGUAGE

Python 3.11+	standard library only	0 dependencies	ThreadingHTTPServer
SHA-256 / MD5	unittest — 50 tests	single-file console	loopback binding

```
case "SIH-2026-014"
source flows = ingest "evidence/netflow.csv"

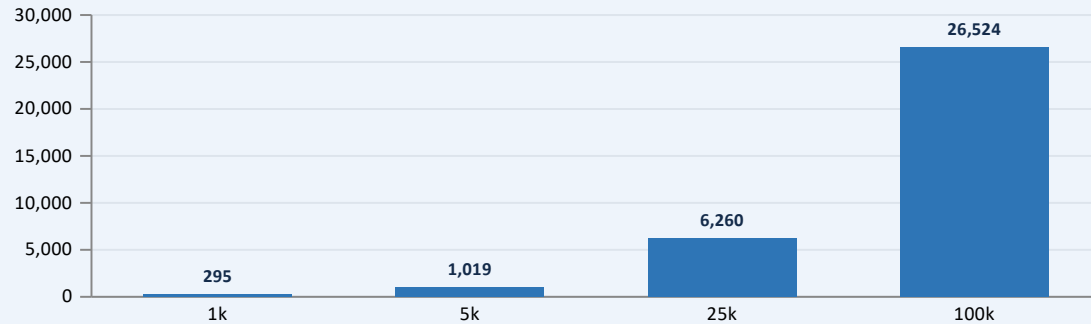
let beacon = flows
|> where (direction == "outbound")
|> where (is_public_ip(dst_ip))
|> where (bytes < 2000)
|> group by (dst_ip, dst_port)
|> sort by (count desc)

finding "Beaconing to an external endpoint"
severity critical from beacon
```

Real script from scripts/beacon_hunt.jky — 332 tokens, 101 AST nodes, 6 findings, 11 ms.

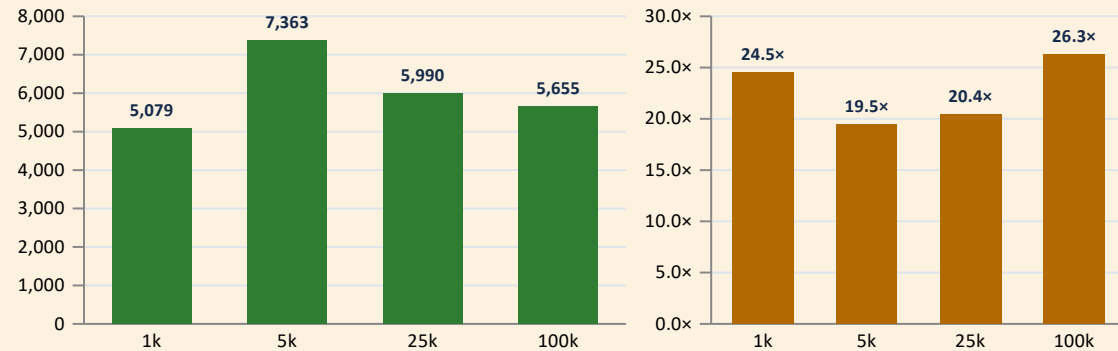
Analysis of the feasibility of the idea · Potential challenges and risks · Strategies for overcoming these challenges

EXECUTION TIME SCALES LINEARLY WITH EVIDENCE VOLUME



One representative investigation — filter, entropy, CTI join, group-by, sort, timeline, 3 findings. Lex + parse stayed at 2.2–3.1 ms at every size, so script cost is independent of data volume.

THROUGHPUT IS FLAT — THE OVERHEAD IS MEASURED, NOT HIDDEN



Throughput holds at 5.1k–7.4k records/sec, so cost is linear and predictable. The tree-walking interpreter costs 20–26× the equivalent hand-written Python — accepted for an auditable reference implementation, and the reason a bytecode VM is the first roadmap item.

All figures measured on the target machine by deck/bench.py and deck/bench_baseline.py — see deck/bench_results.json. Ratios and throughputs, not absolute speeds, are the portable result.

MEASURED RESULTS

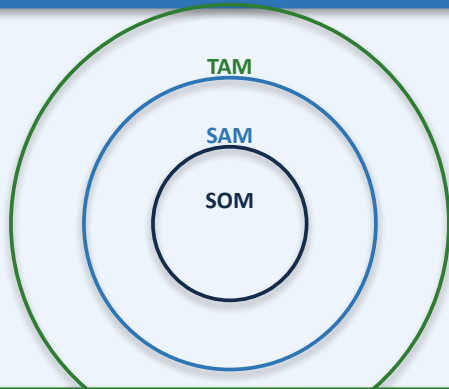
Measured result	Value
Lex + parse	2.2 – 3.1 ms — independent of evidence size
Interpreter throughput	5,079 – 7,363 records / second
Peak interpreter memory	1.9 MB @1k → 188 MB @100k, linear
Overhead vs hand-written Python	20 – 26 ×
Result stability across scales	identical findings at every size
Third-party dependencies	0 — standard library only
Automated test suite	50 tests, all passing

RISKS AND MITIGATION STRATEGY

Risk	Mitigation strategy
Interpreter too slow for full-disk images	Measured and stated, not hidden. Scope scripts to triage sets (10^4 – 10^5 records) where first-response calls are made; bytecode VM is the next milestone.
Examiner over-trusts an automated finding	Severity is authored by the examiner, never inferred. Every finding cites its result set and record count; the report is reproducible.
AV/EDR blocks the collector	Not answered with obfuscation: signed reproducible builds with published hashes for vendor allowlisting, documented telemetry sources only, read-only access.
Evidence altered during analysis	Sources opened read-only and SHA-256 digested at open; nothing is ever written back to a source system.
Console reachable from the network	Loopback binding plus per-request client-address validation; fleet mode is opt-in, over TLS with pinned certificates.
Script path traversal	Resolved path must remain inside scripts/; rejected, and covered by the test suite.

IMPACT AND BENEFITS

ADDRESSABLE IMPACT



TAM

~US\$10–16 B by 2030

Global DFIR — Every organisation that must investigate a breach or hold evidence. Software + services.

SAM

~US\$0.3–0.5 B

Indian obligation-holders — Cyber cells, CERT-In empanelled auditors, MSSPs and university labs with a legal duty to handle evidence.

SOM

~US\$20–40 M

Budget-constrained labs — District cyber cells and university forensic labs on air-gapped workstations, where a free offline tool wins on day one.

SWOT ANALYSIS

S · STRENGTHS

- Zero dependencies — runs offline, air-gapped
- Findings are citable by grammar, not by habit
- 50 unit tests; deterministic, reproducible runs
- Loopback-only console, read-only evidence access

W · WEAKNESSES

- Tree-walking interpreter: ~6k records/sec (measured)
- 14 builtins — no EVT / USN / MFT adapter yet
- Single-host console; no central case management
- Correlation is signature-based, not behavioural

O · OPPORTUNITIES

- DPDP Act 2023 and CERT-In norms create evidence-handling duties
- Sigma / ATT&CK mapping is a natural next layer
- University curricula need a teachable forensic language
- Edge and IoT forensics need small, offline tooling

T · THREATS

- Commercial suites bundled into lab contracts
- Free incumbents (plaso, Hayabusa, Velociraptor) already trusted
- Language adoption is slow without a script-sharing community
- Evidence formats change with every OS release

IMPACT

SOCIAL

A small cyber cell can defend its conclusion in court, and a citizen whose device was seized can see the reasoning behind the finding. Forensic conclusions stop being an expert's private judgement.

ECONOMIC

Removes a paid-tool dependency for labs that cannot license commercial DFIR suites, and removes re-work: one reviewable, cited script replaces a chain of throwaway scripts re-validated per case.

INDUSTRY

Gives DFIR a shared, versionable language. Investigation scripts become portable artefacts that can be peer-reviewed and exchanged between agencies — what Sigma did for detection rules.

WHO THIS REACHES FIRST

- District cyber cells and state CID units.** Small teams, mixed caseloads, no budget for per-seat commercial licences — and a legal duty to document what they did.
- CERT-In empanelled auditors and MSSPs.** Repeatable scripts turn a per-engagement method into a reviewable, transferable asset.
- University and police forensic labs.** An air-gapped workstation is the normal environment, not the exception — and a readable language is teachable.
- The examiner herself.** The console shows the query behind every finding, so a conclusion can be checked rather than taken on trust.

AT A GLANCE

- 0** — licence cost per seat; nothing to renew.
- Air-gapped** — no network calls; the console binds loopback.
- Cited** — every finding names the query that raised it.
- Read-only** — evidence is digested at open, never modified.

Sizing is a top-down estimate from published DFIR market analyses, not a vendor forecast; the method and assumptions are recorded in the repository README.

Details / links of the reference and research work

STANDARDS AND WORKS IMPLEMENTED OR CITED

1.	NIST SP 800-86	Guide to Integrating Forensic Techniques into Incident Response — the collection / analysis / reporting workflow the case model follows.
2.	RFC 3227	Guidelines for Evidence Collection and Archiving — order of volatility governs what JOCKY ingests first.
3.	ISO/IEC 27037:2012	Identification, collection, acquisition and preservation of digital evidence.
4.	NIST SP 800-61r2	Computer Security Incident Handling Guide — the triage-first workflow triage.jky implements.
5.	MITRE ATT&CK	Technique vocabulary the findings map to (T1059, T1071, T1070.001).
6.	Sigma	Prior art for a portable, shareable detection language; JOCKY is the forensic analogue.
7.	Velociraptor VQL	Closest existing art — a query language for endpoint DFIR; JOCKY adds cited findings and reproducible reports.
8.	plaso · Chainsaw · Hayabusa	Timeline and EVTX tooling the timeline stage is measured against.
9.	E. Zimmerman's tools	De-facto reference implementations for EVTX, USN journal and MFT parsing — the adapter roadmap.
10.	Shannon (1948)	A Mathematical Theory of Communication — basis of the entropy() builtin that separates base64 from prose.
11.	FIPS 180-4	Secure Hash Standard — SHA-256 for every evidence digest and script hash.
12.	Python Software Foundation	Standard library only; deliberately no third-party runtime dependency.

Works JOCKY implements against, or deliberately departs from — no result in this deck depends on an uncited external claim.

ARTEFACTS

jockey/ lexer.py · parser.py · runtime.py · builtins.py · cli.py · serve.py

console/index.html operator console — seven tabs, no external assets, no build step

scripts/*.jky beacon_hunt · triage · insider_usb · telemetry_audit

evidence/ one coherent synthetic incident on FIN-WS-014, 2026-09-21

Repository and demo links: (paste before upload) — the whole prototype runs from one checkout with no install step.

RESEARCH CONTRIBUTION & CORE FINDINGS

- **Provenance can be a grammar rule, not a discipline.** Requiring a finding to name the result set that raised it makes an uncited conclusion a parse error rather than a bad habit — the contribution this work claims is that formulation.
- **The interpreter envelope is measured.** 5.1k–7.4k records/sec, linear memory, 20–26× over hand-written Python — which bounds the right workload to triage sets and names the next milestone.
- **Compilation cost is volume-independent.** Lex + parse held at 2.2–3.1 ms from 10^3 to 10^5 records, separating script complexity from data volume as cost centres.
- **“Without triggering security solutions”, read defensibly.** A tool that hides from EDR is indistinguishable from malware to the endpoint's own evidence. Compatibility engineering — signed builds, published hashes, vendor allowlisting, documented telemetry — is the reading that survives an audit.
- **Forensic text breaks conventional lexing.** Windows paths, regex and base64 force a minimal escape set; the textbook escape table silently turned \Temp into a tab. Found by test, fixed by design.

RESEARCH METHODOLOGY & EVOLUTION PIPELINE



Prototype, benchmark harness, evidence set and test suite are all in the submitted repository.